쿼리도(Quoridor) 게임 AI 설계 보고서

1. 문제 분석

1. 쿼리도 특징 분석

쿼리도를 탐색하기 위한 방법을 고민하기에 앞서, 쿼리도라는 게임이 어떠한 종류에 해당하는지, 그 특징을 우선적으로 파악해보았다.

우선 플레이하는 시점에 얻을 수 있는 정보에 따라 나누면 완전 정보 게임과, 불완전 정보 게임으로 나눌 수 있다. 완전 정보 게임은 모든 플레이어가 게임판의 상황, 상대방의 패 등의 모든 정보를 100% 알고 있는 상태에서 플레이하는 게임으로 체스나 바둑, 쿼리도 등이 있다. 반대로 불완전 정보 게임은 상대의 패나 가려진 맵 등 알 수 없는 숨겨진 정보가 존재하는 게임으로 포커 등이 있다.

게임이 결정론적인지 확률적인지에 따라서도 분류할 수 있다. 주사위나 무작위 카드 뽑기 같은 운 요소가 존재하는 경우에는 확률적 게임에 해당하며, 이 경우에는 확률에 의해 예측이 불가능한 상황이 발생한다. 반면 운 요소가 없는 결정론적인 게임의 경우 플레이어가 어떠한 행동을 할 경우 그 결과는 이론적으로 100% 예측 가능하다.

게임의 플레이 방식에 따라 순차적, 동시적 게임으로도 나눌 수 있다. 흔히 턴제 게임이라고도 불리는 순차적 게임은 한 명씩 차례대로 번갈아 가며 행동하는 게임으로 각 플레이어는 상대의 행동을 보고 자신의 행동을 계산할 시간이 주어진다. 반면 가위바위보나 실시간 전략 게임과 같은 동시적 게임의 경우 모든 플레이어가 동시에 행동을 결정하기 때문에 상대의 행동을 바탕으로 자신의 행동을 결정할 수 없다.

마지막으로 게임의 경쟁 구조에 따라 분류할 수도 있다. 흔히 제로섬 게임이라도 불리는, 자신이 얻는 이득이 상대의 손실이 되는 극단적인 경쟁 구조를 가진 게임의 경우 자신의 이득이 최대가 되도록 자신만을 고려하여 플레이할 수 있다. 반대로 협력이나 동맹 등 자신만의 이득이 게임

플레이의 승리와 직결되지 않은 비제로섬 게임의 경우 앞선 제로섬 게임에서의 전략이 최선이 아닐 수 있다.

이러한 게임 분류를 바탕으로 했을 때, '쿼리도'라는 게임은 완전 정보, 결정론적, 순차적, 제로섬 게임에 해당한다. 이는 바둑, 체스, 오목과 같은 종류에 속하는 것이다. 쿼리도에서는 게임에 참여하는 두 플레이어가 서로 같은 보드를 보면서 진행하기 때문에 자신과 상대의 벽 개수, 상대의 위치, 상대가 설치한 벽 등 모든 정보가 공개된다. 즉, 숨겨진 정보가 존재하지 않는다. 쿼리도는 말을 움직일 방향이나 벽을 설치할 위치를 플레이어가 직접 결정할 수 있기 때문에 주사위와 같이 확률적으로 결정되는 요소가 아예 없다. 그리고 쿼리도는 각 플레이어가 번갈아가며 말을 움직이거나 벽을 설치하는 수를 두를 두는 턴제 게임이기 때문에 상대의 행동을 바탕으로 자신의 다음 행동을 결정할 수 있는 순차적 게임이다. 또한 게임의 규칙상 먼저 반대편에 도착한 사람이 무조건 승리하기 때문에 무승부가 없으며, 한 플레이어가 승리하면 상대는 패배하는 형태이기 때문에 제로섬 게임에 해당한다.

2. AI가 탐색할 상태공간

쿼리도를 단순히 말을 움직이는 게임으로 생각할 경우 탐색할 경우의 수가 매우 적다고 생각할 수도 있으나, 벽 설치의 경우 맵 어디에든 설치가 가능하기 때문에 경우의 수가 매우 많아지게 된다. 2006년 P.J.C. Mertens의 논문 "A Quoridor-playing Agent"에 따르면 쿼리도의 상태 공간의 크기는 3.9905 x 10^42 으로 2개의 말이 배치되는 자리 조합과 두 플레이어가 가진 20개의 벽을 128개의 벽 설치 가능 영역에 대해 불가능한 벽 설치를 배제하고 설치할 수 있는 최대 조합 범위를 수학적으로 계산하여 얻은 값이다. 또한 평균 분기 계수와 평균 게임 턴수를 기반으로 추정한 게임 트리 크기의 경우 10^162 으로 매우 크다는 것을 알 수 있다. 이처럼 상태공간이 크고 복잡한 쿼리도와 같은 게임에서는 벽과 같은 요소가 당장의 승률 하락보다는 중/장기적인 손해를 불러오기 때문에 각 수에 대한 깊은 탐색을 요구되고, 각 탐색 깊이에서

뻗어나가야 하는 가지수가 많을 경우 탐색 깊이 확보에 불리하게 작용하여 승률 저하가 나타날 수 있다. 탐색 깊이가 깊어질수록 이미 설치된 벽으로 인해 이론적 상태공간의 크기는 점점 작아지지만, 상태공간의 크기가 거의 균일하다고 가정한다면 탐색 깊이에 따라 탐색할 경우의 수가 지수적으로 증가하는 상황에서 탐색 깊이를 최소 7~10 이상 확보하기 위해서는 각 깊이에서의 분기 수를 8~10 이내로 유지해야 한다고 생각하였다. 이에 따라 각 탐색 깊이에서 실질적으로 모든 가능한 벽의 경우의 수를 포함할 수 없다고 생각하였고, 탐색이 필요한 벽들의 위치를 잘 선별하는 것이 중요할 것이라고 분석하였다.

2. 상태 표현

쿼리도의 상태를 표현하기 위해서는 각 플레이어의 말의 위치와 설치된 벽의 위치를 저장해야 한다. 이때 저장된 상태는 탐색 과정에서 매우 빈번하게 사용되기 때문에 쓰기보다는 읽기에 최적화된 방법으로 저장해야 한다.

만약 벽의 위치를 개별적으로 리스트의 형태로 저장하게 된다면 수 탐색 과정에서 해당 위치에 벽이 존재하는지 판단하기 위해서는 O(N)의 시간 복잡도가 소요된다. 즉, 주어진 history를 그대로 사용하게 될 경우 탐색이 비효율적이다. 이에 따라 맵을 2차원 배열로 표현하고, 각 위치에 대해 플레이어 존재 여부, 벽 설치 여부를 저장하는 방식으로 결정하였다.

이때 각 위치에 대한 플레이어 존재 여부, 벽 설치 여부를 따로 저장하거나 튜플로 묶어 저장하게 될 경우 메모리가 낭비된다. 왜냐하면 저장해야 하는 정보는 true, false로 표현될 수 있는 binary 정보인데 반해 이를 나눠서 저장할 경우 메모리 시스템이 byte 단위로 할당할 수 밖에 없는 한계로 인해 1bit를 표현하기 위해 1byte를 사용하게 된다. 이를 막기 위해 bitmasking을 통해 여러 정보를 하나의 정수로 표현하는 방식을 사용하였다.

이를 위해 9 x 9 크기의 2차원 리스트를 만들었고, 각 칸의 정수에 대해 0번 비트는 플레이어 말의 존재 여부, 1번~4번 비트는 상하좌우 방향의 벽의 존재 여부를 기록하였다. 이러한 2차원 배열 구조 자체는 게임 규칙을 적용하기에는 직관적이었으나 추후 탐색 알고리즘 내부에서 수읽기를

위해 판 전체를 가상으로 복사하여 사용할 때마다 리스트의 동적 할당 오버헤드가 예상보다 커서 문제가 발생하였다.

MCTS 탐색 트리 내부에서 초기 v1~v3에서는 앞서 언급했던 방식대로 리스트와 현재 상태를 dictionary에 모두 담은 뒤 통째로 deepcopy를 통해 복사하는 스냅샷 방식을 사용하였다. 하지만 이 방식은 매 탐색마다 무거운 쿼리도 상태 dict의 생성을 유발하면서 성능 저하를 유발하였고, 탐색 루프 내에서 시뮬레이션을 돌리는 과정에서 deepcopy로 인한 과도한 메모리 garbage 발생으로 인해 GC(Garbage Collector)의 과부하를 유발하였다. 이로 인해 실질적으로 탐색하는 노드의 수가 매우 작아지는 문제가 발생하였다.
이를 개선하기 위해 v4~v6에서는 중복 탐색을 막기 위한 transposition table을 도입하였다. 이때 해당 노드가 이미 탐색되었는지를 확인하기 위해 현재 보드의 상태를 튜플로 묶어서 해당 튜플을 해싱한 값을 기준으로 확인하는 방식을 사용하였다. 이 방식을 도입하여 캐시 적중을 통해 중복 탐색을 방지하여 탐색의 효율을 높일 수 있었지만, 여전히 2차원 리스트를 기반으로 매 상태 갱신마다 리스트를 튜블에 타입을 변환하여 넣는 과정에서 오버헤드가 발생하여 탐색 병목이 여전히 발생하였다.
이를 근본적으로 해결하기 위해 v7과 v8에서부터는 MCTS 탐색 트리 내부에서 2차원 리스트를 사용하지 않도록 하였고, 8개의 정수로 묶인 비트보드를 구현하여 상태 정보를 복사하는 비용을 2차원 리스트 복사에서 정수 8개 복사로 대폭 감소시켰다. 또한 플레이어의 위치도 0~80 사이의 정수로 저장하도록 변경하고, 벽이 + 모양으로 교차하는 것을 막아주는 피벗 점유 정보 또한 64비트 정수 마스크로 저장하여 최적화하였다.

3. 알고리즘 설계

1. v1 - MCTS 도입

v1에서는 MCTS(Monte Carlo Tree Search)를 도입하였다. 이는 트리 탐색 알고리즘 중 하나로 정확한 확률 분포를 구하기 어려울 때 무작위 표본 추출을 통해 확률 분포를 도출하는 Monte

Carlo 알고리즘을 트리 탐색에 응용한 알고리즘이다. 트리 탐색 알고리즘에는 MCTS 말고도 Minimax Tree Search나 Alpha-Beta Pruning과 같은 다른 알고리즘들도 존재한다. 그럼에도 MCTS를 선택하게 된 이유는 쿼리도의 특성과 연관되어 있다.

우선 Minimax Tree Search의 경우 체스나 바둑, 쿼리도처럼 각 플레이어가 번갈아 가며 수를 두는 형태의 게임에서 상대방이 최선의 수를 둔다는 가정 하에 탐색을 진행하는 알고리즘이다. 상대방이 최선의 수를 둔다고 가정했기에 그 상대의 수가 자신에게 최소한의 영향을 미치도록 탐색한다. 그래서 Minimax에서는 자신의 차례에서는 가능한 후보들 중 가장 평가함수의 값이 높은 값이, 상대의 차례에서는 평가함수의 값이 가장 낮은 값이 선택된다. 이러한 방식을 사용하면 알고리즘 자체는 상대가 최선의 선택을 하더라도 최선의 대응을 할 수 있도록 설계되어 있기 때문에 평가함수의 품질만 좋다면 높은 성능을 얻을 수 있다. 그럼에도 이러한 Minimax를 사용하지 않은 이유는 쿼리도가 체스와 같은 게임들보다 훨씬 많은 분기수를 가지고 있기 때문이다. 그렇기 때문에 쿼리도와 같은 게임에서는 트리 내에서 모든 경로를 탐색하는 것이 아닌, 가지치기를 해가며 탐색하는 Alpha-Beta Search나 확률적으로 노드를 평가하여 탐색하는 MCTS를 사용해야 한다고 생각하였다. 쿼리도의 벽 설치의 경우의 수가 방대하기 때문에 확률적 접근이 필요할 것이라고 생각하여 MCTS를 선택하게 되었다.

MCTS에서는 확률적 무작위 시뮬레이션을 기반으로 아래와 같이 트리를 확장한다.

1. 선택 (Selection) : 루트 노드에서 시작하여, 승률과 탐사 사이의 균형을 맞춰주는 UCB1(Upper Confidence Bound) 수식을 사용해 가장 유망한 자식 노드들을 선택한다.
2. 확장 (Expansion) : 선택된 리프 노드가 아직 시도해보지 않은 경로가 있다면 새로운 자식 노드를 추가하여 트리를 확장한다.
3. 시뮬레이션 (Simulation / Rollout) : 새롭게 확장된 노드에서 게임의 승패가 결정될 때까지 두 플레이어가 무작위로 수를 두는 가상의 게임을 진행하여 결과를 도출한다.

4. 역전파 (Backpropagation) : 시뮬레이션으로 얻은 승패 결과를 트리를 따라 거슬러 올라가면서 방문했던 모든 부모 노드들의 통계를 갱신한다.

MCTS는 기본적으로 확률 분포를 계산할 때 모든 노드를 직접 탐색하지 않고 일부 표본을 추출하여 사용하기 때문에 MCTS가 계산한 확률과 실제 확률은 차이가 발생할 수 있다. 하지만 여기에 확률이 높을 것으로 예상되는 노드 쪽으로만 트리의 확장이 일어난다면 잠재력이 있었지만 탐색이 일어나지 않아 그냥 묻히는 수들이 발생한다. 그렇기 때문에 MCTS에서는 각 노드를 평가할 때 UCB1을 사용하여 단순히 승률만을 보는 것이 아니라 그 노드가 선택된 횟수도 함께 고려하도록 설계하여 이러한 문제를 방지한다.

```
def ucb1(self, parent_visits):
    if self.visits == 0:
        return INF
    return self.value() + C_PUCT * math.sqrt(math.log(parent_visits) / self.visits)
```

[UCB1 공식 구현]

v1에서는 이러한 정석적인 MCTS 구조를 그대로 게임에 적용하였다. 매 탐색마다 게임 상태를 모두 dictionary에 담아 복사하고 게임이 끝날 때까지 무작위 행동을 반복하는 rollout을 수행하였다. 하지만 쿼리도의 특성상 승패가 결정될 때까지 최소 수십 턴 이상의 무의미한 수들이 이어졌고, 이 뿐만 아니라 시뮬레이션을 위해 보드를 준비하는 과정에서 사용되는 dictionary deepcopy로 인해 발생하는 복사 오버헤드가 GC에 부하를 일으키는 등의 문제가 발생하였다. 결과적으로 5초 안에 고작 50가지의 수도 탐색하지 못하는 수준의 심각하게 낮은 성능을 보였다. 이를 통해 쿼리도에서는 경우의 수가 너무 방대한 벽 설치라는 것이 존재하기 때문에 단순히 무작위 rollout에 의존하는 탐색은 부적합하다는 것을 알게 되었다.

2. v2~v3

rollout의 한계를 알게 된 후 MCTS에서 각 노드를 평가할 때 rollout의 결과가 아닌 휴리스틱 평가함수를 사용하도록 구조를 변경하였다. 이를 위해 최단 거리를 기반으로 한 평가함수를

도입하였다. 하지만 평가함수를 통한 휴리스틱 탐색에도 불구하고 트리 확장 단계에서 병목이 발생하였다. 당시 알고리즘은 쿼리도에서 가능한 모든 128개의 벽을 후보로 두어 가상의 벽을 세워보고 최단경로가 존재하는지 모두 확인한 뒤 탐색을 진행하였다. 이 과정에서 실질적인 탐색보다 탐색을 준비하기 위해 벽을 세우는 과정에서 더 많은 연산량을 요구하게 되었고, 알고리즘이 유의미한 탐색 깊이를 도달하지 못하게 되었다.

이를 해결하기 위해 우선 128개로 매우 많았던 벽 설치 부호군을 축소시키는 필터링을 적용하였다. 전체 보드를 검사하는 것이 아니라, 전진하려는 최단 경로의 인근 좌표들만 벽 배치 후보 생성 과정에서 고려함으로써 한 턴에 고려할 분기를 20여개 수준으로 감소시켰다.
이 뿐만 아니라 탐색 과정에서 매 node 전환마다 보드의 상태를 snapshot으로 복사하고 붙여넣는 과정에서 발생하는 부하와 병목을 막기 위해 백트레킹을 도입하였다. 즉, 새로운 벽을 가상으로 놓아볼 때마다 매번 보드 상태 전체를 통째로 복사하는 것이 아니라, 현재 단일 보드 위에서 벽을 임시로 설치해보고 유효성을 검사한 후 다시 원래 상태로 복구시키는 방식을 사용하였다. 이러한 최적화를 통해 트리에서 새로운 노드를 탐색할 때마다 발생하던 병목을 어느정도 해소할 수 있었으며, 연산 속도를 5~10배 이상 향상시킬 수 있었다.

3. v4 ~ v6

v4부터는 중복 탐색을 줄이기 위한 최적화를 진행하였다. 현재 MCTS는 트리의 형태로 분기해나가며 경로들을 탐색하는데 이전까지 지나온 경로들은 다르더라도 결과적인 상태가 같다면 그 두 노드의 향후 탐색 결과는 같을 것이라는 아이디어에서 출발하게 되었다. 예를 들어 어떠한 상태 A에서 플레이어 1과 2가 서로를 견제하기 위해 말을 움직였는데, 만약 이후의 상태 B에서 결과적으로 두 플레이어가 상태 A에서 있었던 위치로 되돌아 왔다면 상태 A와 상태 B는 지나온 과거 상태는 다르지만 결과적으로 이후 탐색할 노드들은 같을 것이다. 그렇기에 같은 종류의 노드가 등장했을 때 이를 중복하여 탐색하는 것이 아니라 이미 탐색한 결과를 가져다가

사용할 수 있도록 하기 위해 한 번 방문한 노드의 정보를 튜플의 형태로 묶어 캐싱하는 전치 테이블(Transposition Table)을 도입하였다. 하지만 이를 도입하게 되면서 상태 공간의 탐색이 단순히 트리의 형태가 아니라 그래프의 형태로 변경되게 되었고, 이로 인해 기존 MCTS에서 각 노드를 평가하였던 수식인 UCB1으로는 유망한 수를 먼저 파고들지 못하는 문제가 발생하였다.

이를 해결하고자 PUCT(Predictor Upper Confidence Trees)라는 새로운 공식을 도입하였다.

```
def puct(self, parent_visits, c_puct=1.414):
    exploration = c_puct * self.prior * math.sqrt(max(parent_visits, 1))
    bias = 0.5 * self.prior / (1 + self.visits)
    if self.visits == 0:
        return exploration + bias
    return self.value() + exploration / (1 + self.visits) + bias
```

[PUCT 공식 구현]

PUCT의 핵심은 사전 확률의 도입이다. self.prior의 형태로 구현된 이 사전확률은 새로운 노드를 탐색하기 전에 휴리스틱하게 계산된 해당 행동에 대한 유망도를 나타낸다.

사전 확률은 다음 3가지를 고려하여 결정된다.

1. 비상 수비 상태

MCTS 노드 확장 시 현재 플레이어의 최단 경로 거리와 상대의 최단 경로 거리를 계산하고, 상대방이 골라인까지 4칸 이내로 접근하였고 자신의 남은 거리가 상대보다 뒤쳐지고 있는 상황이라면 즉시 비상 수비가 필요한 상태로 판정한다.

2. 이동 행동의 사전 확률

이동 가능한 유효 인접 위치들에 대해 최단 경로 BFS를 바탕으로 다음과 같이 점수를 배정한다.

최단 경로를 단축하는 방향 : 자신의 최단 거리를 줄여주는 이동에 대해서는 사전 확률을 0.7로 설정하여 탐색 초반에 우선적으로 탐색할 수 있도록 유도한다.

최단 경로를 단축하지 않는 방향 : 자신의 최단 거리를 단축시키지 않는 불필요한 이동에 대해서는 사전 확률을 0.3으로 설정하여 탐색의 중요도를 낮춘다.

비상 수비 활성화 : 이동보다 상대방을 가로막는 수비벽이 절대적으로 중요하기 때문에 이동 행동의 모든 사전 확률을 10배 낮게 각각 0.07과 0.03으로 설정한다.

3. 벽 설치 행동의 사전 확률

각 탐색 깊이마다 놓을 수 있는 벽 후보 위치를 모두 탐색하는 것을 막기 위해 유의미한 사전 확률 값을 가진 벽의 목록을 아래와 같은 조건으로 필터링하여 선별한다.

1. 상대 최단 경로를 직접 차단하는 벽

상대의 최단 경로 상의 인접 엣지를 벽으로 가로막을 수 있는 벽을 선별한다. 이때 일반적으로는 사전 확률을 0.6으로 부여하지만, 비상 수비 상황에서는 사전 확률을 5.0으로 올려 차단벽을 놓는 경우의 수에만 집중할 수 있도록 한다.

2. 간접 위협 예방 벽

상대에게 남은 벽이 있는 상황에서 자신의 최단 경로 중 앞부분 4칸 중 상대방이 벽을 놓아 경로를 방해할 위험이 있는 좌표들을 계산하고, 해당 좌표들이 실제로 놓였을 때 자신의 최단 경로를 2칸 이상 증가시키는 심각한 위협이라면 그 벽을 상대가 설치하지 못하도록 미리 교차벽으로 막는 벽을 선별한다. 이 경우에는 사전확률을 0.8로 설정한다.

3. 무의미한 벽

위 두 조건에 부합하지 않는 무의미한 벽의 경우 사전 확률을 0.0으로 설정하여 MCTS가 탐색하지 못하도록 한다. 즉 후보 목록에서 제거된다.

4. v7 ~ v8

쿼리도의 상태를 표현하는 방식의 구조적 병목으로 인해 탐색의 속도가 저하된다는 것을 알게 된 후 이를 비트보드로 변경하였다. 자세한 내용은 앞선 **2. 상태 표현** 에서 이미 언급했기 때문에 생략한다.

한편, 방어막을 설계하는 과정에서 초반에는 모든 확장 노드마다 자신의 경로를 위협하는 수비벽을 계산하려고 하였으나 연산량의 큰 증가로 인해 상대가 골라인에 근접한 위기 상황에서만 수비벽을 탐색하도록 변경하였다.

4. 평가 함수

1. 무작위 rollout 시뮬레이션 (v1)

초기 v1에서는 평가함수를 사용하지 않고 무작위 롤아웃 시뮬레이션을 통해 해당 노드를 승리 또는 패배로 평가하였다. 이러한 방식은 휴리스틱한 평가함수 대비 해당 노드를 정확히 평가할 수 있다는 장점이 있었지만, 쿼리도와 같이 상태공간이 매우 큰 게임에서는 무작위 rollout 시뮬레이션이 제한 시간 내에 평가를 완료하지 못하기 때문에 이 방식을 폐기하고 휴리스틱 평가함수를 설계하게 되었다.

2. 평가함수 (v2 ~ v6)

rollout을 대체하기 위해 설계한 평가함수는 쿼리도의 판세를 최대한 잘 표현하기 위해 3가지 요소가 결합된 형태이다.

1. 상대적 최단 경로 비율 (dist_ratio) : 상대 최단경로 / (자신 최단경로 + 상대 최단경로) 로 계산하여 자신의 최단경로가 상대의 최단경로보다 짧을수록 값이 1에 수렴한다.

2. 보유 벽 비대칭성 (wall_asymmetry) : (자신 벽 개수 - 상대 벽 개수) / 10 으로 계산하여 남은 벽을 고려했을 때의 자신의 우위를 평가한다.

3. 목표 지점까지의 진행도 (mover_progress) : max(0, 1 - 내 최단경로 / 8) 로 계산하여 자신의 말이 목표 지점 라인까지 얼마나 다가갔는지를 평가한다.

초기에는 아래와 같이 이 3가지 요소를 단순히 가중치를 통해 선형 결합하여 하나의 평가함수 값으로 계산하였다.

score = dist_ratio + w_1 * wall_asymmetry + w_2 * mover_progress

이후 v6에서는 양 플레이어가 목적지 2칸 앞까지 접근했을 때 보너스 또는 패널티를 주는 로직을 추가하였다. 하지만 이러한 방식에도 근본적인 문제점이 존재하였다.

예를 들어 경기 후반에 자신의 최단경로가 1이고, 상대의 최단경로가 2인 상황의 경우 상대적 최단 경로 비율의 값이 2 / 3 = 0.66.. 이 된다. 하지만 초반에 자신의 최단경로가 8, 상대의 최단경로가 9인 상황에서는 9 / 17 = 0.52.. 가 된다. 두 상황 모두 자신이 상대를 1칸 앞서고 있는 동일한 우세 상태임에도 불구하고, 전자의 가치가 후자보다 훨씬 높게 책정되는 문제가 발생한다.
이러한 상대적 최단 경로 비율의 왜곡은 AI의 판단 오류를 발생시킨다. 경기 초중반에는 AI가 한 수를 잘 두어서 상대보다 1칸 앞서나가는 유리한 방향으로 나아가더라도 dist_ratio는 0.50에서 0.52로 매우 조금 바뀌게 된다. 이로 인해 AI는 1칸 앞서나가는 수보다 오히려 벽을 아끼는 수들을 우선적으로 탐색하여 상대를 견제하는 수들을 제대로 탐색하지 않게 되고, 초반에 주도권을 주기 위해 상대를 견제하고 자신의 길을 확보하지 못하여 상대의 역전을 허용하게 된다.
경기 후반에는 dist_ratio의 왜곡으로 인해 거리 1칸을 앞서는 가치의 변동폭이 0.16으로 급격하게 증가하면서 결과적으로 남은 벽의 개수를 전혀 고려하지 않고 무지성으로 전진만을 시도하다가 최종적으로 상대의 수비벽에 막혀 패배하게 된다.

이처럼 단순히 평가함수에 포함된 한 요소의 비율식의 왜곡이 성능 저하를 만들게 되었고, 이러한 왜곡이 본 AI에서 더 크게 나타났던 이유는 기존 MCTS와 달리 rollout 없이 평가함수에만 의존하는 형태로 변형된 MCTS를 사용해서 각 노드들의 평가함수 의존성이 크게 증가한 상태였기 때문이라고 생각한다.
또한 평가함수를 여러 요소들의 선형결합으로 만들었는데, 각 요소들을 결합하는 가중치를 항상 일정하게 둔 것도 영향을 주었다고 생각한다.

이 뿐만 아니라 v3에서부터 도입한 것 중에 Dynamic BFS가 있다. MCTS 탐색에서 각 노드의 가치를 평가하거나 벽의 유효성을 검증할 때마다 전체 격자판에 대해 BFS를 매번 처음부터 다시 수행하는 기존 방식은 심각한 병목을 유발하였다. 이를 개선하기 위해 BFS의 결과를 저장해두었다가 맵의 수정이 발생하면 BFS의 결과를 부분적으로 갱신하는 방식의 Dynamic BFS를 도입하였다.

Dynamic BFS는 부분적 거리 갱신을 기반으로 동작한다.

부분적 거리 갱신

새로운 벽이 설치되었을 때 격자판 전체를 처음부터 다시 탐색하는 것이 아닌, 새로 놓인 벽에 의해 경로가 단절된 엣지들의 영향을 역으로 추적하고, 기존 최단 거리 맵에서 이 끊어진 엣지에 의존하고 있던 경계선 노드들 중 우회 경로를 가진 노드들을 시작점으로 삼아 Dijkstra를 수행하여 영향권 아래의 노드의 거리만을 재계산한다.

이 Dynamic BFS를 통해 매 벽 설치 테스트마다 발생하던 최단 거리 검증 병목을 해결할 수 있었다.

3. 평가함수 (v7)

앞에서 사용한 비율 방식의 문제를 해결하기 위해 v7에서는 상대와의 절대적인 거리 차이만을 사용하도록 수정하였다. 이로써 최단경로의 차이는 게임의 어느 단계에서나 동일하게 평가되게 되었다.

이와 함께 남은 벽을 단순히 선형적으로 비교하는 것이 아니라, 벽 자원의 한계에 따른 영향을 반영하여 감쇠 법칙을 적용한 점수 계산을 도입하였다. 10개의 벽을 가지고 있는 상태에 1를 소모하는 것과 1개의 벽만 남은 상태에서 마지막 벽을 소모하는 것의 가치는 매우 큰 차이가 있다고 생각했기 때문이다. 이는 다음과 같이 점수를 계산하였다.

```
wall_values = {0:0, 1:15, 2:25, 3:32, 4:37, 5:41, 6:44, 7:46, 8:48, 9:49,
10:50}
wall_bonus = wall_values.get(my_walls) - wall_values.get(opp_walls)
```

그리고 v7에서는 여기에 더해 초근접전에서의 페널티와 보너스를 추가로 계산하여 더하는 로직을 도입하였었는데, 단순히 페널티와 보너스를 더하게 되면서 기존 MCTS에서의 값과 값 분포가 다른 값이 더해지는 문제가 발생해 탐색 과정이 불안정해지는 문제가 발생하였다.

4. 평가함수 (v8)

앞선 v7에서의 문제를 해결하기 위해 시그모이드 함수를 통해 0과 1 사이의 부드러운 승률 기댓값 확률 범위로 변환하였다. 하지만 단순히 시그모이드 함수만을 사용한다면 시그모이드 함수의 고질적 문제인 기울기 소실 문제가 발생한다. 예를 들어 자신의 위치가 8칸 이상 우세하여 확실하게 이긴 경우 시그모이드 함수의 출력값이 1에 수렴하며 기술이가 0에 수렴한다. 이렇게 되면 MCTS는 9칸 앞서 이기는 수와 8칸 앞서 이기는 수 사이의 가치 차이를 구분하지 못하게 되고, 최선의 수를 두지 못하고 의미없는 수를 두며 방황하는 문제가 발생하게 된다.
이러한 문제를 없애기 위해 시그모이드 함수에 선형 보정 항을 가중합으로 결합하였다.

최종적으로 평가함수는 다음과 같다.

x = (opp_dist - my_dist) * 10 + wall_bonus + Proximity_Bonus

Score = 0.8 * Sigmoid(x) + 0.2 * Linear(x)

Linear(x) = 0.5 + max(-0.49, min(0.49, x / 200))

이러한 평가함수의 개선을 통해 평가함수에 대한 의존성이 큰 변형된 형태의 MCTS가 높은 성능을 낼 수 있도록 만들 수 있었다.

5. 시간 관리

1. Search Fail Fallback

시간 초과를 방지하기 위해 agent 함수가 호출되었을 시점의 시간을 time 라이브러리를 이용해 기록하고, 매 탐색 루프마다 시간을 확인함으로써 탐색에 사용한 시간이 3.5초(시간제한을 안전하게 설정한 경우) 또는 4.5초(시간제한을 조금 도전적으로 설정한 경우) 을 넘어갔다면 그 즉시 탐색을 중단하게 된다.

구현한 탐색 알고리즘이 시간 내에 탐색에 실패하거나 오류가 발생할 경우를 대비해 fallback 함수를 통해 보완하였다. fallback 함수는 호출한 탐색 함수에서 오류가 발생하여 try-catch 문에서 오류가 잡히거나 설정된 제한 탐색 시간이 지났음에도 탐색이 완료되지 않은 경우 호출된다. fallback 함수의 역할은 탐색이 실패한 상황에서 빠른 시간 내에 수를 판단하여 제출함으로써 몰수패를 방지하는 것이다.

6. 소스코드 주요 부분

1. 비트보드 병렬 BFS

본 코드는 81칸 격자의 1단계 최단거리 확산을 CPU 시프트 연산 4회로 병렬 완수하는 쿼리도 Bit BFS의 핵심 코드이다.

```
def compute(self, blocked_up, blocked_down, blocked_right, blocked_left,
player_positions):
    BOARD = self._BOARD_MASK

    frontier = self._goal_frontier
    visited = frontier

    self.dist_map = [self.INF] * 81
    fy = self.finish_line_y
    for x in range(9):
        self.dist_map[x + fy * 9] = 0

    d = 1
    while frontier:
        # 상하좌우 비트 경계 마스크와 차단벽 논리합 결합
        can_up = frontier & ~self._TOP_EDGE & ~blocked_up
        can_down = frontier & ~self._BOT_EDGE & ~blocked_down
        can_right = frontier & ~self._RIGHT_EDGE & ~blocked_right
```

```
        can_left = frontier & ~self._LEFT_EDGE & ~blocked_left

        # 병렬 시프트 및 미방문 칸 논리곱 전파
        new_frontier = ((can_up << 9) | (can_down >> 9) | ((can_right << 1)
& BOARD) | (can_left >> 1)) & ~visited & BOARD

        if not new_frontier:
            break

        visited |= new_frontier
        frontier = new_frontier

        # LSO (Least Significant One) 기반 거리맵 갱신
        tmp = new_frontier
        while tmp:
            bit = tmp & (-tmp)
            self.dist_map[bit.bit_length() - 1] = d
            tmp ^= bit
        d += 1

    for px, py in player_positions:
        if self.dist_map[px + py * 9] == self.INF:
            return False
    return True
```

2. 전치 테이블 기반 MCTS 트리 재활용

아래 코드는 MCTS의 선택->확장->평가->역전파 과정을 구현한 코드이고, 전치 테이블을 통해 이미 계산한 노드를 재활용하고, 중첩 재귀를 통해 새 루트 하위로 도달 가능한 자식 노드 상태만 수집하여 도달 불가능한 노드들을 정리하는 로직도 포함되어 있다.

```
def search(self, root_state, time_limit=0.2):
    # 1수 앞 즉시 승리/차단용 즉시 개입 루프
    decisive = self.get_decisive_move(root_state)
    if decisive is not None:
        return decisive

    # 루트 노드 전치 테이블 재활용 및 메모리 Pruning
    root_hash = root_state
    if root_hash in self.transposition_table:
        root = self.transposition_table[root_hash]

        # 도달 가능한 자식 노드만 선별 캐싱
        reachable = {}
        visited_nodes = set()
```

```
        def collect(node):
            if id(node) in visited_nodes:
                return
            visited_nodes.add(id(node))
            reachable[node.state] = node
            for child in node.children.values():
                collect(child)
        collect(root)
        self.transposition_table = reachable
    else:
        root = MCTSNodeBitwise(root_state)
        self.transposition_table = {root_hash: root}

    start = time.time()
    cnt = 0

    while True:
        if cnt & 63 == 0 and time.time() - start >= time_limit:
            break

        node = root
        search_path = [node]
        visited_ids = {id(node)}

        # 선택 (Selection)
        while node.is_fully_expanded() and node.children:
            best_uct = -1.0
            best_child = None
            for child in node.children.values():
                uct = child.puct(node.visits)
                if uct > best_uct:
                    best_uct = uct
                    best_child = child
            if best_child is None or id(best_child) in visited_ids:
                break
            visited_ids.add(id(best_child))
            node = best_child
            search_path.append(node)

        # 확장 (Expansion)
        node_state = node.state
        if not (node_state[0] // 9 == 8 or node_state[1] // 9 == 0):
            if node.untried is None:
                node.untried = MCTSBitwise._get_actions(node_state)

            if node.untried:
                act_info = node.untried.pop()
```

```
                act = act_info[0]
                next_state = MCTSBitwise._apply_action_bitwise(node_state,
act)

                if next_state in self.transposition_table:
                    child = self.transposition_table[next_state]
                else:
                    child = MCTSNodeBitwise(next_state, action=act,
prior=act_info[-1])
                    self.transposition_table[next_state] = child

                node.children[act] = child
                node = child
                search_path.append(node)

        # 평가 (Evaluation)
        val = MCTSBitwise._evaluate(node.state)

        # 역전파 (Backpropagation)
        for n in reversed(search_path):
            n.visits += 1
            n.value_sum += val
            val = 1.0 - val

        cnt += 1

    best_act = max(root.children.items(), key=lambda x: x[1].visits)[0]
    return best_act
```

3. 비트마스킹을 통한 행동 반영

아래 코드는 비트마스킹을 통해 빠르게 탐색하고자 하는 행동(action)을 맵에 반영하는 코드이다.

```
@staticmethod
def _apply_action_bitwise(state, action_idx):
    pos0, pos1, wcnt0, wcnt1, wall_h, wall_v, turn, pivot_mask = state
    next_turn = 1 - turn

    # 플레이어 이동 행동
    if action_idx < 81:
        dest = action_idx
        if turn == 0:
            return (dest, pos1, wcnt0, wcnt1, wall_h, wall_v, next_turn,
pivot_mask)
        else:
            return (pos0, dest, wcnt0, wcnt1, wall_h, wall_v, next_turn,
pivot_mask)
```

```
    # 가로벽 설치 행동
    elif action_idx < 162:
        idx = action_idx - 81
        new_wall_h = wall_h | (1 << idx)
        wx, wy = idx % 9, idx // 9
        pivot = wy * 8 + wx
        new_pivot = pivot_mask | (1 << pivot)
        if turn == 0:
            return (pos0, pos1, wcnt0 - 1, wcnt1, new_wall_h, wall_v,
next_turn, new_pivot)
        else:
            return (pos0, pos1, wcnt0, wcnt1 - 1, new_wall_h, wall_v,
next_turn, new_pivot)
    # 세로벽 설치 행동
    else:
        idx = action_idx - 162
        new_wall_v = wall_v | (1 << idx)
        wx, wy = idx % 9, idx // 9
        pivot = (wy - 1) * 8 + wx
        new_pivot = pivot_mask | (1 << pivot)
        if turn == 0:
            return (pos0, pos1, wcnt0 - 1, wcnt1, wall_h, new_wall_v,
next_turn, new_pivot)
        else:
            return (pos0, pos1, wcnt0, wcnt1 - 1, wall_h, new_wall_v,
next_turn, new_pivot)
```

4. 벽 겹침 확인

새로운 벽을 놓을 때 가로벽, 세로벽이 겹치는지 확인하고, 현실에서 불가능한 +자로 맞추어진 벽 또한 확인하는 코드이다.

```
def _wall_geom_ok(wall_h, wall_v, pivot_mask, walls_left, wx, wy, d):
    if walls_left <= 0:
        return False

    # 가로벽 충돌 검사
    if d == 0:
        if not (0 <= wx <= 7 and 0 <= wy <= 7):
            return False
        bit = 1 << (wx + wy * 9)
        if wall_h & (((bit >> 1) & ~MCTSBitwise._COL8) | bit | (bit << 1)):
            return False
        pivot = wy * 8 + wx
    # 세로벽 충돌 검사
    else:
```

```
        if not (0 <= wx <= 7 and 1 <= wy <= 8):
            return False
        bit = 1 << (wx + wy * 9)
        if wall_v & ((bit >> 9) | bit | (bit << 9)):
            return False
        pivot = (wy - 1) * 8 + wx

    # 가로/세로벽이 중심점 피벗을 서로 교차하는 +자 벽 검사
    return not (pivot_mask & (1 << pivot))
```

7. 실험 결과

아래 실험 결과는 **로컬 환경**에서 진행되었으며, **싱글스레드 실행**을 기준으로 한다.

버전에 따른 agent 성능 개선

agent 버전	평균 NPS (초당 탐색 수	0.2초당 탐색 횟수	최대 탐색 깊이
v1	15	3	3
v2	1,200	240	3
v3	3,500	700	6
v5	8,500	1,700	9
v7	4,500	900	5
v8	55,000	11,000	16

1. 단순 무작위 agent

1.1 동일 엄격 조건

조건 : 양 agent 동일 0.2초 제한

결과 : 10전 10승 0패 (승률 100%) / 오류 및 실격 0회

1.2 핸디캡 조건

조건 : 본 agent만 0.2초 제한, 상대 agent 5.0초 제한

결과 : 10전 10승 0패 / 오류 및 실격 0회

1.3 동일 여유 조건

조건 : 양 agent 동일 5.0초 제한

결과 : 10전 10승 0패 (승률 100%) / 오류 및 실격 0회

2. Greedy agent

2.1 동일 엄격 조건

조건 : 양 agent 동일 0.2초 제한

결과 : 10전 10승 0패 (승률 100%) / 오류 및 실격 0회

2.2 핸디캡 조건

조건 : 본 agent만 0.2초 제한, 상대 agent 5.0초 제한

결과 : 10전 10승 0패 (승률 100%) / 오류 및 실격 0회

2.3 동일 여유 조건

조건 : 양 agent 동일 5.0초 제한

결과 : 10전 10승 0패 / 오류 및 실격 0회

3. agent_2101 (6월 26일 기준 agent_2101이기 때문에 업데이트된 agent_2101과의 결과는 차이가 있을 수 있다)

3.1 동일 엄격 조건

조건 : 양 agent 동일 0.2초 제한

결과 : 10전 10승 0패 (승률 100%) / 오류 및 실격 0회

3.2 핸디캡 조건

조건 : 본 agent만 0.2초 제한, 상대 agent 5.0초 제한

결과 : 10전 10승 0패 (승률 100%) / 오류 및 실격 0회

3.3 동일 여유 조건

조건 : 양 agent 동일 5.0초 제한

결과 : 10전 10승 0패 (승률 100%) / 오류 및 실격 0회

4. agent_2103 (6월 22일 기준 agent_2103이기 때문에 업데이트된 agent_2103과의 결과는 차이가 있을 수 있다)

4.1 동일 엄격 조건

조건 : 양 agent 동일 0.2초 제한

결과 : 10전 8승 2패 (승률 80%) / 오류 및 실격 0회

4.2 핸디캡 조건

조건 : 본 agent만 0.2초 제한, 상대 agent 5.0초 제한

결과 : 10전 7승 3패 (승률 70%) / 오류 및 실격 0회

4.3 동일 여유 조건

조건 : 양 agent 동일 5.0초 제한

결과 : 10전 9승 1패 (승률 90%) / 오류 및 실격 0회

이처럼 로컬 환경에서는 높은 성능을 보였다.

하지만 CPU 성능이 제한된 colab 환경에서는 큰 성능 저하를 나타냈다.

(colab에서의 실험은 시간 부족 문제로 인해 진행하지 못했다. colab에서 업데이트된 agent_2101을 상대로 몇번 실행해보았을 때에는 승률이 그렇게 높지는 않았던 것 같다.)

8. 한계점

1. 긴급 방어 시스템의 수동적 활성화

현재 구현되어 있는 긴급 방어 시스템은 상대방이 골라인까지 4칸 이내로 접근했을 때에만 작동하는, 즉 수동적으로 활성화되는 구조이다. 그렇기 때문에 만약 상대방이 게임 초중반부터 여러 수 앞을 바라보고 장기적인 길막기나 가두기 등의 전술을 펼친다면 현재의 AI는 이를

비상상황으로 인식하지 않는다. 물론 기본적인 MCTS 탐색으로 어느정도 대비가 가능하겠지만, 해당 전술만을 특화시킨 정책 기반 전술 특화 AI를 상대로는 보다 일반론적인 광범위한 탐색을 수행하는 MCTS가 상대적으로 불리해질 수 있다.

2. 복합 벽 트랩에 대한 시야 상실

플레이어가 4칸 이상 멀리 떨어져 있을 때, 3~4개의 벽이 연쇄적으로 설치되어 완전히 가둬지는 장기적인 가두기 전략의 경우 롤아웃이 즉시 가치 평가 함수로 단축되면서 탐색 트리의 리프 노드에서 제대로 평가되지 않을 수 있다. 이로 인해 실제로는 매우 위험한 상황임에도 이를 평가함수가 제대로 반영하지 못하여 제때 대응하지 못하는 문제가 발생할 수 있다. 또한 리프 노드의 휴리스틱 평가함수는 현재 시점의 단순 거리 차이만을 가지고 판단하므로 8수 이상 뒤에나 서서히 완전히 닫히는 정밀하고 장기적인 가두기 전략을 탐색 초기에 깊게 예측해내지 못하는 한계가 존재한다. 이 한계는 쿼리도의 초반 탐색 과정에서 분기할 경우의 수가 다른 게임들보다 많기 때문으로 MCTS나 Alpha-Beta Pruning과 같은 트리 탐색 기반 알고리즘들은 초반부터 해당 AI가 특화 전략을 펼치는 것을 감지하기에 구조적으로 어려움이 있다고 생각한다.

하지만 앞선 한계점에도 불구하고 트리 탐색 기반 알고리즘을 설계한 이유는 시간 제한이 조금 더 넉넉한 상황에서는 더 정밀한 탐색을 수행하는 MCTS나 Alpha-Beta Pruning과 같은 트리 탐색 알고리즘들이 성능상 더 우위를 점하기 때문이다. 5초라는 시간 제한이 복잡하고 정밀한 MCTS가 단순하고 빠른 정책 기반 전략 특화 AI를 넘는 그 임계점보다는 약간 짧은 시간이었던 것 같고, 이를 설계 초반에 고려하지 못한 것이 아쉽게 느껴진다. (로컬에서 실행한 결과에서는 colab CPU 대비 더 높은 성능의 CPU의 영향으로 그 임계점을 넘은 것처럼 확인되었지만, colab에서 지는 것을 보았을 때에는 실행 환경의 차이가 큰 것 같다)

9. 개선 방안

1. 동적 트리 확장

탐색 과정에서 매 탐색 깊이에서 같은 개수의 분기를 탐색하는 것이 조금은 비효율적일 수 있다고 생각하였다. 너무나도 뻔한 정답 수가 존재하는 경우에는 적게 탐색하고, 깊이 생각해야 할 경우에는 많은 경우의 수를 탐색하는 동적 트리 확장을 도입한다면 이를 개선해볼 수 있을 것이라 생각한다. 이를 활용하면 탐색이 끝난 후 남는 시간에 다음 상대의 수를 예상하여 다음 턴의 탐색을 미리 수행해둠으로써 진짜로 복잡하고 싶은 탐색이 필요한 시점에 조금이나마 시간을 벌어 더 나은 수를 둘 수 있도록 하는 방식의 로직도 적용해볼 수 있을 것 같다.

2. 로컬 실행과 colab 실행 사이의 성능 괴리 해결

앞서 여러 분석을 통해 문제를 어느정도 파악하긴 했으나 로컬에서의 결과와 colab에서의 결과가 무시할 수 없을 수준으로 매우 큰 차이가 발생하고 있다. 그렇기에 이 문제가 현재 개발한 AI의 정확히 어느 부분으로 인해 발생하는지를 더 정확히 분석하고 프로파일링하여 성능 괴리 문제를 해결해볼 것이다.

10. 소스코드 전체

Colab 링크:

https://colab.research.google.com/drive/1mcHUO3BdHGiE1wUmqbqwybwQnCAqkykE?usp=sharing

11. 출처

[1] Kocsis, L., & Szepesvári, C. (2006). *Bandit based Monte-Carlo Planning*. ECML.

[2] Rosin, C. D. (2011). *Multi-armed bandits with episodic reward*. Annals of Mathematics and Artificial Intelligence.